

Applying GPUs to SUSP Search

Minh Phuc Nguyen

Department of Computer Science
Union College
Schenectady, New York, USA
`nguyenp@union.edu`

Abstract. We follow the Cohn–Umans approach [8,7], studying an algebraic object called a strong uniquely solvable puzzle (SUSP) and its application to matrix multiplication. The goal is to find sufficiently large SUSPs to reduce the matrix multiplication exponent ω in $O(n^\omega)$. We build on the work of Anderson and Le [4], focusing on a subclass of SUSPs known as simplifiable SUSPs. Their staged search framework relies mainly on CPU multithreading, while their earlier CUDA simplification path processes one puzzle at a time and does not lead to practical end-to-end search speedup on medium target sizes. We introduce a batched GPU simplification path and integrate it into staged iterated local search so that whole batches of puzzle simplifications are coordinated inside the real search loop. In our benchmarks, the batched search increases total simplification throughput by roughly $71\times$ to $284\times$ on the measured targets.

Keywords: matrix multiplication · strong uniquely solvable puzzle · arithmetic complexity · upper bounds · reduction · application · general purpose GPU computing · heterogeneous computing

1 Introduction and previous work

Given two matrices A of size $m \times k$ and B of size $k \times n$, the matrix $C = A \times B$ of size $m \times n$ can be calculated using the formula

$$C_{ij} = \sum_{a=1}^k A_{ia} B_{aj}, \quad i \in \{1, \dots, m\}, j \in \{1, \dots, n\}$$

The optimization of matrix multiplication has long been a major research topic, with continuous improvements in both theoretical methods and hardware capabilities. From a theoretical perspective, research focuses mainly on reducing the matrix multiplication exponent ω in the complexity $O(n^\omega)$. The value of ω has been improved from 3 in the naive algorithm to approximately 2.8074 with the Strassen algorithm [22], then to approximately 2.376 with the Coppersmith–Winograd algorithm [9], followed by many further works [10,23,16,2,11]. Currently, the smallest known theoretical bound for ω is 2.371339 [1]. From a hardware perspective, the trend is to increase the throughput of computations per

second, measured in FLOPS (floating-point operations per second), with support from hardware such as GPUs [18], TPUs [13], and application-specific integrated circuits (ASICs) [15].

We focus on reducing theoretical complexity, specifically by lowering ω as discussed above. We follow the framework of Henry Cohn and Christopher Umans [8,7], concentrating on an algebraic object called a SUSP. We search for SUSPs to derive better upper bounds on the matrix multiplication exponent ω . For a fixed width k , finding SUSPs with larger size s (number of rows) leads to stronger bounds on ω .

Verifying whether a puzzle is a SUSP under the original definition is known to be in coNP, and prior general verification methods rely on reduction to 3D matching [3], which is NP-hard [14]. As a result, this gives a theoretically exponential-time procedure and, in practice, requires heavy solvers such as MIP or SAT.

Anderson and Le [4] introduce a new subclass of SUSPs, called simplifiable SUSPs, designed to allow polynomial-time verification.

1.1 Simplifiable SUSPs

Following Cohn et al. [7], an (s, k) -puzzle is a subset of words over the alphabet $\{1, 2, 3\}$.

Definition 1 (Puzzle). For $s, k \in \mathbb{N}$, an (s, k) -puzzle is a subset $P \subseteq [3]^k$ with $|P| = s$. We call s the size of P and k the width of P .

A puzzle can be viewed as a 2D array where each entry takes one of three possible values: 1, 2, or 3.

Definition 2 (Uniquely Solvable Puzzle (USP)). An (s, k) -puzzle P is uniquely solvable if for all $\pi_1, \pi_2, \pi_3 \in \text{Sym}_P$, either $\pi_1 = \pi_2 = \pi_3$, or there exist a row $r \in P$ and column $c \in [k]$ such that at least two of

$$(\pi_1(r))_c = 1, \quad (\pi_2(r))_c = 2, \quad (\pi_3(r))_c = 3$$

hold.

Definition 3 (Strong USP (SUSP)). An (s, k) -puzzle P is strong uniquely solvable if for all $\pi_1, \pi_2, \pi_3 \in \text{Sym}_P$, either $\pi_1 = \pi_2 = \pi_3$, or there exist a row $r \in P$ and column $c \in [k]$ such that exactly two of

$$(\pi_1(r))_c = 1, \quad (\pi_2(r))_c = 2, \quad (\pi_3(r))_c = 3$$

hold.

From this definition, verifying a general SUSP requires checking all permutations of the rows, making direct verification extremely computationally expensive.

Cohn et al. show the following connection between strong USPs and upper bounds on the exponent of matrix multiplication.

Lemma 1 ([7, Corollary 3.6]). *Let $\epsilon > 0$. If there is a strong uniquely solvable (s, k) -puzzle, then there is an algorithm for multiplying n -by- n matrices in time $O(n^{\omega+\epsilon})$ where*

$$\omega \leq \min_{m \in \mathbb{N}_{\geq 3}} \left(\frac{3 \log m}{\log(m-1)} - \frac{3 \log s!}{s \cdot k \log(m-1)} \right).$$

For each puzzle P , we construct a tripartite 3-uniform hypergraph H_P . Its vertex set consists of three labeled copies of the rows of P :

$$P_1 = \{u^{(1)} \mid u \in P\}, \quad P_2 = \{v^{(2)} \mid v \in P\}, \quad P_3 = \{w^{(3)} \mid w \in P\}.$$

For any $u, v, w \in P$, define

$$f(u, v, w) = 1 \iff \exists i \in [k] \text{ such that exactly two of } u_i = 1, v_i = 2, w_i = 3 \text{ hold.}$$

The edge set of H_P is defined by

$$E(H_P) = \{(u^{(1)}, v^{(2)}, w^{(3)}) \in P_1 \times P_2 \times P_3 \mid f(u, v, w) = 0\}.$$

Then, a puzzle P is a SUSP if and only if the 3-uniform hypergraph H_P has no nontrivial perfect matching; that is, there does not exist

$$\begin{aligned} M &\subseteq E(H_P), \\ \bigcup_{e \in M} e &= P_1 \cup P_2 \cup P_3, \\ e \cap e' &= \emptyset \quad \text{for all } e \neq e' \in M. \end{aligned}$$

with

$$M \neq \{(u^{(1)}, u^{(2)}, u^{(3)}) \mid u \in P\}.$$

In other words, verifying a SUSP reduces to checking whether there is a *non-trivial* perfect matching in H_P [3]. However, the 3-dimensional perfect matching problem on hypergraphs is a highly complex combinatorial problem (NP-hard in general) [14]. Therefore, although the reduction to H_P avoids explicitly enumerating all row permutations, SUSP verification still requires searching a very large combinatorial space of candidate matchings, causing the computational cost to grow rapidly with the puzzle size s .

To address this, the simplifiable SUSP subclass is introduced. The main idea is not to directly search for a perfect matching, but instead to iteratively reduce H_P while preserving the set of perfect matchings. Specifically, H_P is gradually reduced to a smaller hypergraph H'_P that has the same set of perfect matchings.

A puzzle P is called a simplifiable SUSP if the hypergraph H_P can be reduced step by step to the trivial matching:

$$M_{\text{trivial}} = \{(u^{(1)}, u^{(2)}, u^{(3)}) \mid u \in P\}.$$

Anderson and Le [4] show that if this reduction process succeeds, then H_P has no nontrivial perfect matching, and therefore P is a SUSP. More importantly, this simplification process can be carried out in polynomial time in s and k , by repeatedly eliminating edges based on matchings of the “2D projections” of H_P . Thus, unlike general SUSP, simplifiable SUSP provides an efficient verification method by constructing a “short proof” through a sequence of hypergraph reduction steps. The following theorem from [4] holds:

Theorem 1. Let $\epsilon > 0$. If there exists a simplifiable (s, k) -SUSP P , then there is an algorithm for multiplying $n \times n$ matrices in time $O(n^{\omega+\epsilon})$, where

$$\omega \leq \min_{m \in \mathbb{N}_{\geq 3}} 3 \cdot \frac{k \log m - \log s}{k \log(m-1)}.$$

This theorem shows that for fixed k , larger values of s lead to smaller ω . In other words, finding larger simplifiable SUSPs implies a lower theoretical complexity for matrix multiplication.

1.2 Verification of Simplifiable SUSPs

Anderson and Le [4] proposed the SIMPLIFY algorithm (Algorithm 1) to reduce H_P to H'_P as described earlier. The algorithm begins by constructing the original hypergraph H_P , then projecting it onto three 2D planes and processing each in turn.

On the current projection R_f , it identifies 2D edges that do not belong to any maximum matching of that projection. For each such edge, when processing R_0 , all 3D edges of the form $(*, u, v)$ are removed; for R_1 , edges of the form $(u, *, v)$ are removed; and for R_2 , edges of the form $(u, v, *)$ are removed.

After removing edges, the hypergraph is reprojected, the algorithm moves to the next plane, and the process repeats until a full pass over all three projections results in no further edge removals. At that point, a fixed point is reached, yielding the fully simplified 3D hypergraph. Anderson and Le [4] also prove that this procedure runs in polynomial time; their analysis gives the running time of SIMPLIFY as $O\left(\frac{s^{5.5}}{\sqrt{\log s}}\right)$ with respect to the puzzle size.

1.3 Challenges in the Simplification Process and SUSP Search

Although the complexity of the SIMPLIFY operation has been reduced to polynomial time instead of exponential time, the cost of this step is still quite large. Constructing and traversing the 3D structure of H_P alone has complexity $O(s^3)$, and SIMPLIFY also has an initialization cost of $\Omega(s^3)$, even when the simplification process stops early after meeting the required condition. More importantly, SIMPLIFY is the main operation inside the hot loop of the entire search program. The program does not verify only one puzzle; instead, it must repeatedly perform this operation on hundreds of millions, or even billions, of puzzles in a raw search space of up to $3^{s \cdot k}$ puzzles.

Algorithm 1 Simplify**Input:** A 3D graph H .**Output:** A fully-simplified 3D graph.

```

1: function SIMPLIFY( $H$ )
2:    $R_0, R_1, R_2 \leftarrow \text{PROJECT}(H)$ 
3:    $f \leftarrow 0$ 
4:    $\text{sinceChange} \leftarrow 0$ 
5:   while  $\text{sinceChange} < 3$  do
6:      $\text{edgesToRemove} \leftarrow \text{CALCEDGESTOREMOVE}(R_f)$ 
7:     for  $(u, v) \in \text{edgesToRemove}$  do
8:       if  $f = 0$  then
9:         Delete all edges  $(*, u, v)$  from  $H$ 
10:      else if  $f = 1$  then
11:        Delete all edges  $(u, *, v)$  from  $H$ 
12:      else if  $f = 2$  then
13:        Delete all edges  $(u, v, *)$  from  $H$ 
14:      if  $\text{edgesToRemove} = \emptyset$  then
15:         $\text{sinceChange} \leftarrow \text{sinceChange} + 1$ 
16:      else
17:         $\text{sinceChange} \leftarrow 0$ 
18:         $R_0, R_1, R_2 \leftarrow \text{PROJECT}(H)$ 
19:       $f \leftarrow (f + 1) \bmod 3$ 
20:   return  $H$ 

```

With such a large search space, brute-force search for SUSPs is not practical for large puzzle sizes. Anderson and Le [4] introduced an iterative local search algorithm that uses a heuristic function. Their method is based on one property of SUSPs: if a puzzle of size (s, k) is a SUSP, then any puzzle of size (s', k) , where $s' \leq s$ and the s' rows form a subset of the original s rows, is also a SUSP [7]. Therefore, their search for a SUSP of size (s, k) starts from a known SUSP of size $(s - 1, k)$. They then add one random row to the base puzzle, gradually modify its elements, evaluate the heuristic value, and move closer to a SUSP of size (s, k) using iterative local search. They use the evaluation function

$$f(P) = s^3 - |E(\text{Simplified}(H_P))|,$$

where H_P is the 3D graph obtained from the puzzle P and $E(\text{Simplified}(H_P))$ is the number of edges remaining in H_P after simplification. If a puzzle is a simplifiable SUSP, then after simplification, H_P contains only the trivial matching, so $f(P) = s^3 - s$. This value is used as the heuristic for the search procedure, with the goal of making $f(P)$ as large as possible and gradually approaching the value $s^3 - s$.

1.4 Limitations of Using the CPU for Simplification and Search

Although SIMPLIFY has high computational cost, it also has strong potential for parallel computation because SIMPLIFY works on each edge of a puzzle independently, so an embarrassingly parallel approach can be used. Additionally, simplifying one puzzle is independent of simplifying other puzzles, so the search process exposes two layers of parallelism. Anderson and Le [4] optimized the search process using OpenMP on the CPU and implemented a CUDA version of SIMPLIFY on the GPU, because many steps follow a SIMD or block-based processing pattern: (i) initializing projections, (ii) SCC decomposition, (iii) computing edges-to-remove, and (iv) updating H . They used a parallel SCC algorithm on CUDA [5] and reported a speedup of about $\sim 10\times$ on a GPU (GTX 1060) compared to one CPU core (i5-6400) for puzzles up to size 196, even without deep optimization [4].

1.5 Limitations of the Previous GPU Algorithm

Even so, the original GPU algorithm still has bottlenecks, which means applying it to the full search problem does not substantially improve over using OpenMP on a CPU-only implementation.

The first issue is that each puzzle is transferred individually between the CPU and GPU. GPUs are optimized for large workloads with minimal synchronization with the CPU. However, the CUDA implementation of Anderson and Le [4] transfers one puzzle at a time from the CPU to the GPU, lets the GPU perform the computation, and then transfers the result back, while still requiring synchronization to ensure correctness. The data transfer cost and the CPU stall time while waiting for the GPU consume a significant amount of time. As a result, the $10\times$ speedup is not fully reflected in the actual search performance.

To measure this effect directly, we compared two versions of the same staged local search. The CPU-only version ran every SIMPLIFY call on the CPU, while the GPU-enabled version kept the original CUDA offload for individual puzzles. All other search settings were identical.

The measurements were run on an AMD Ryzen 9 9900X CPU (12 cores/24 threads) with an NVIDIA GeForce RTX 5070 Ti GPU (16 GiB). As shown in Figure 1, enabling GPU simplification changed end-to-end search throughput by only $0.96\times$, $0.99\times$, $1.03\times$, and $1.02\times$ for target sizes (14, 6), (23, 7), (35, 8), and (52, 9), respectively. The isolated per-puzzle GPU simplification speedup does not translate into a comparable staged-search speedup.

The second issue is that memory allocation is performed inside the main search loop. Memory allocation is expensive and time-consuming on both sides of the host-device boundary. Placing all of these allocations inside the main loop is wasteful, since the memory can be reused after one piece of data has finished being processed. Moreover, this creates additional internal synchronization, which prevents overlap between different processing stages. We discuss this problem in more detail in the batching design below.

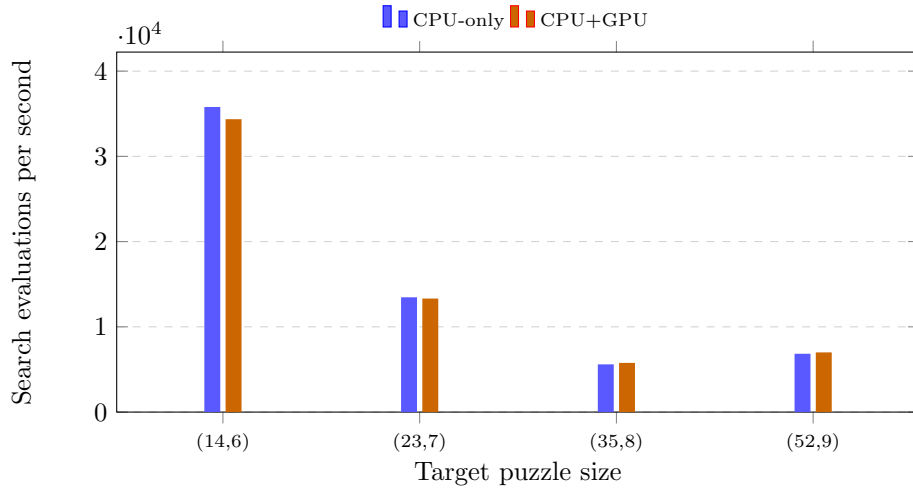


Fig. 1: End-to-end throughput of the original staged search program with simplification forced to the CPU or with GPU simplification enabled.

2 Batch-Based GPU Simplification

This section shows that, when many puzzles are grouped together and processed in batches, GPU throughput, or the number of puzzles simplified per second, can be much higher than the throughput of the CPU algorithm, even when OpenMP is used to exploit the available CPU threads. In our experimental environment, the largest isolated simplification test in Figure 5 reaches about $218\times$ the throughput of the CPU implementation once the batch size reaches 128.

2.1 Batched Candidate Evaluation and Serialized Puzzle Layout

The enhanced program first groups compatible candidate puzzles before invoking SIMPLIFY. Each group contains only puzzles with the same width and height, and each batch is fixed to contain at most 256 puzzles. This number was chosen empirically and can be tuned for different research environments.

The puzzles are then serialized into a one-dimensional array of packed elements. The unnecessary parts of the `Puzzle` objects are removed, leaving only the raw content. This content is then encoded and compressed further. Since each puzzle element has only three possible values, 1, 2, and 3, it can be represented with two bits. In this work, we only consider puzzles with width at most 15. Therefore, each row of a puzzle can be packed into one 64-bit word. Thus, 256 puzzles are compressed into one array: rows are stored consecutively within each puzzle, and puzzles are stored consecutively within the batch, as shown in Figure 2. Each element can be accessed through an array index and bit-mask operations. This reduces the amount of data that must be transferred between host

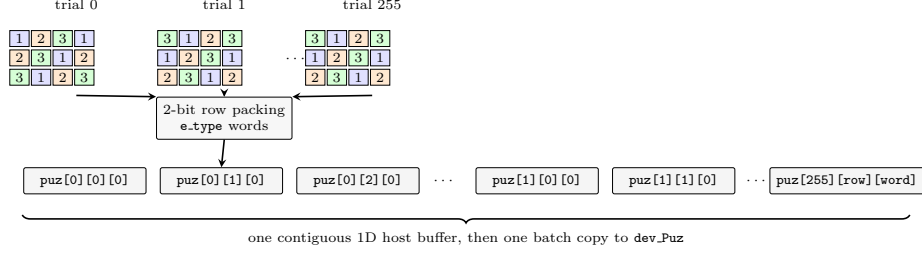


Fig. 2: Batched serialization of candidate puzzles. Puzzles with common s and k are packed row-by-row into 64-bit words and concatenated into one 1D buffer addressed by trial, row, and word.

memory and device memory, which can otherwise introduce significant latency and affect program performance.

2.2 Persistent GPU Resources

The device-side memory layout follows the same principle. Rather than maintaining an array of high-level puzzle objects on the GPU, the enhanced implementation stores the major data fields in contiguous logical regions. Each trial receives a disjoint slice of the packed puzzle buffer. This organization is close to a struct-of-arrays layout: data of the same kind are stored together, while the trial index selects the slice belonging to one puzzle. The layout is especially useful for projection and reduction kernels, where consecutive threads scan consecutive entries of the 3D graph and where each trial’s projection can be cooperatively loaded into shared memory. Figure 3 illustrates this partitioning of one persistent allocation into per-trial logical buffers.

Device allocation and deallocation are expensive relative to a small simplification, and they can also introduce synchronization points. The enhanced implementation therefore allocates one persistent `GpuResources` object per GPU before the search loop begins. Each resource owns one CUDA stream, pinned host buffers for serialized puzzle input and integer results, and one large device allocation. Pinned host memory supports efficient asynchronous transfers, and CUDA devices with copy engines can overlap host-to-device transfers, device-to-host transfers, and kernel execution [20]. This is the mechanism used later to overlap batch transfer with simplification work.

2.3 Device-Side Convergence State

At the algorithmic level, SIMPLIFY repeatedly removes edges from H_P by cycling through the three projections R_0, R_1, R_2 . On the current face, the algorithm computes which projected edges cannot belong to any maximum matching of that projection, removes the corresponding TDM entries, and then advances to the next face. The process stops when three consecutive faces produce no change;

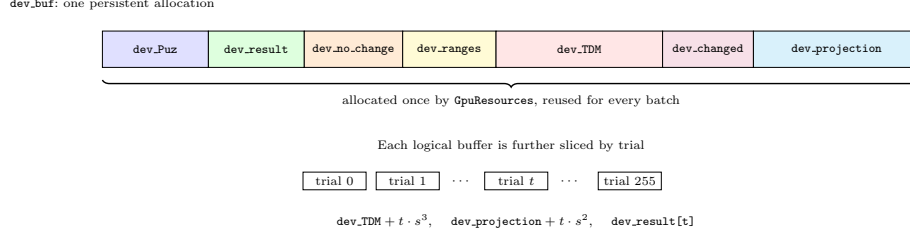


Fig. 3: Persistent GPU resource layout. A single device allocation is partitioned into logical buffers for packed puzzles, 3D graph states, projections, convergence flags, range labels, and per-trial results.

equivalently, a full cycle over R_0, R_1, R_2 , up to rotation, has failed to delete any edge.

The original CUDA path keeps this stopping logic on the host. After each face, the device writes a single `disChanged` flag, the host copies it back, and the CPU decides whether to launch the next face. The program works properly, but it creates a synchronization point after every face of every puzzle. For a workload composed of many small independent simplifications, those synchronization points are a major part of the total running time.

In our enhanced CUDA implementation, the decision to stop or continue is moved to the GPU-side code. Each trial keeps its own convergence state, so the host no longer needs to copy a stopping flag back after every face. In our experiments, we observed that the SIMPLIFY process converges after at most seven passes. For small puzzle sizes, with width around 6 or 7, the process usually converges after about four or five passes. Therefore, we fix the number of SIMPLIFY passes for all puzzles at 12, which provides a safety margin, although 9 passes was already sufficient in all experiments with puzzles of width at most 10. On the host side, we launch the kernels for SIMPLIFY sequentially, as shown in Figure 4, and run a fixed 12-pass schedule over faces 1, 2, 3, 1, 2, 3, ...

The disadvantage of this method is that the host does not stop launching the fixed schedule as soon as the SIMPLIFY stopping condition has been reached. However, at the beginning of each launched kernel, the kernel checks the trial’s convergence state. If the desired result has already been reached, the kernel returns immediately and performs no work, so the extra cost is small. The advantage of this method is that there is no synchronization point during the SIMPLIFY process. This is useful when batched simplification is applied to the search program. During the search, the host side can copy the whole batch to device memory once, enqueue the fixed kernel schedule, then continue with other work and return only to receive the final result. At that point, the GPU runtime controls the order of the queued work, leaving the CPU free to perform other parts of the search. The important point is that removing synchronization allows CPU and GPU work to overlap and improves performance.

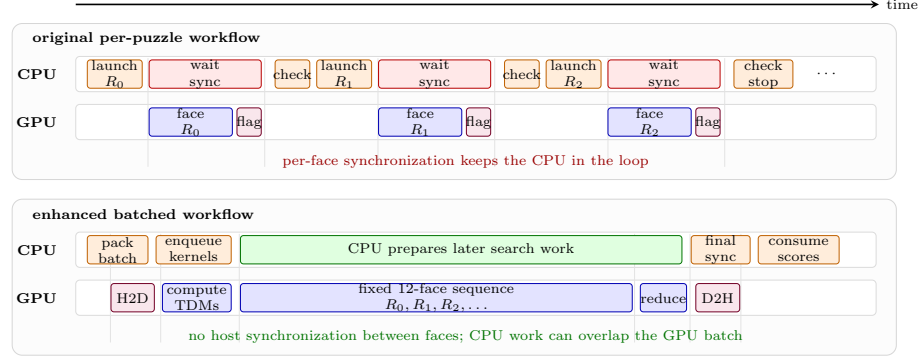


Fig. 4: Original and enhanced CUDA workflows. The original path synchronizes with the host after each face; the enhanced path enqueues a fixed device-side sequence and returns only the final batch result.

2.4 Kernels and GPU-Side Reductions

In the CUDA implementation of Anderson and Le [4], each call to `SIMPLIFY` processes only one puzzle. The main kernels are `compute_TDM`, which constructs the 3D graph from the puzzle; `calc_projection`, which computes the projection onto each face; `fb_kern`, which is the main kernel used to simplify each 2D face; and `modify_TDM`, which updates the 3D graph after edges have been cut on a face. The program then uses summation kernels to count the remaining elements in the 3D graph. In this design, the GPU repeatedly launches small kernels for one puzzle at a time, while the CPU still participates in the control loop and checks the stopping condition.

In our enhanced batched implementation, the kernels are rewritten in a batch-based form. As described in Subsection 2.1, all data have already been serialized into one-dimensional arrays. Each thread is assigned a `global_idx`. This index is then split into `trial` and `local_idx`: `trial` identifies which puzzle in the batch the thread is processing, while `local_idx` identifies the corresponding position inside that puzzle’s 3D graph.

This change appears clearly in the kernel that computes the projection onto each face and the kernel that updates the 3D graph. In the original CUDA implementation, the implementation uses a three-dimensional grid (x, y, z) for a single puzzle. In our enhanced batched implementation, the kernels use one-dimensional indexing and map the local index back to (x, y, z) inside each `trial`. Each puzzle’s data are indexed so that different puzzles do not overwrite one another.

Another change is in the main trimming kernel. In the original CUDA implementation, `fb_kern` runs one block for one puzzle and reads the projection directly from global memory. In our enhanced batched implementation, each block corresponds to one `trial`, and the projection of that trial is loaded cooperatively into dynamic shared memory before the adjacency list is built. Shared memory has much lower latency than global memory, so when the threads in

the same block repeatedly access the projection to find neighbors and reverse neighbors, the data are closer to the streaming multiprocessor. This is especially suitable for the reachability step, because the same projection matrix is read many times while the SCC labels are updated.

The kernel that constructs the 3D graph from the puzzle is also optimized. The original CUDA implementation uses the puzzle as a character string, and each GPU thread must loop over the columns and read the characters '1', '2', and '3'. Our enhanced batched implementation uses the encoded puzzle representation described in Subsection 2.1. The kernel uses bit masks to check many columns at once: each puzzle value is encoded with two bits, and XOR, shift, and AND operations create masks for the conditions that the first row is 1, the second row is 2, and the third row is 3. For $k < 16$, an entire row fits in one word, so the loop becomes a very short sequence of bitwise operations. This type of optimization fits the GPU well because bitwise operations have low latency, use few branches, and can run uniformly across many threads.

The final reduction is a major difference between the original CUDA implementation and our enhanced batched implementation. In the original CUDA implementation, after simplification finishes, the program counts the remaining edges in the 3D graph with two kernels: `sum_z` sums along the z -dimension to create an intermediate 2D array, and `sum_y` then sums along the y -dimension to create a 1D array. This 1D array is copied back to the CPU, and the CPU performs the final summation. This approach creates an additional intermediate buffer, another kernel launch, an extra device-to-host transfer, and still leaves the CPU involved in a reduction task that is well suited to the GPU.

Our enhanced batched implementation replaces this whole reduction stage with a templated `sum` kernel parameterized by `BLOCK_THREADS` and using `cub::BlockReduce` from CUB [19]. Each block processes one part of a 3D graph inside one `trial`. Each thread sums a sequence of elements, and CUB then performs the block-level reduction in shared memory. The result of each block is accumulated into the final result with `atomicAdd`. For each `trial`, the program uses a fixed number of blocks, currently four blocks per trial, so multiple blocks can participate in counting one 3D graph. Thus, the final result of each puzzle is produced directly on the GPU and is copied back to the host only once after the whole batch has finished.

The data flow becomes cleaner: the 3D graph is created on the GPU, simplified on the GPU, counted on the GPU, and the CPU receives only the final result array. This reduces synchronization, reduces memory transfer, and avoids pulling the CPU into small computations that are repeated many times. In the context of a batch of 256 puzzles, these optimizations give the GPU enough work to hide memory latency and maintain better occupancy.

2.5 Benchmark

To isolate the effect of batching, we benchmarked average simplification time per puzzle as a function of batch size. The benchmark compares three paths: the CPU implementation with OpenMP, the original per-puzzle CUDA implementation,

and the enhanced batched CUDA implementation. Figure 5 reports time, so lower values are better.

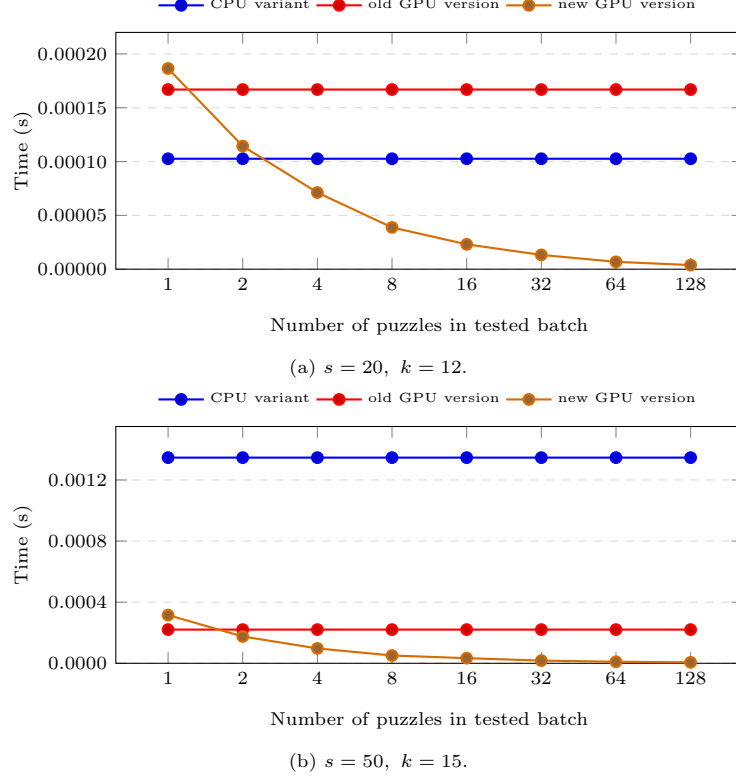


Fig. 5: Average time to simplify one puzzle as batch size increases. The enhanced GPU path is slow for a single small puzzle because it still pays fixed GPU overhead, but the per-puzzle cost drops sharply once those costs are amortized across a batch.

For $s = 20, k = 12$, the enhanced batched path starts at 1.86×10^{-4} seconds per puzzle for a single trial, slower than both the CPU baseline (1.03×10^{-4}) and the original per-puzzle CUDA path (1.67×10^{-4}). The small single-trial case does not provide enough work to amortize CUDA launch and transfer overhead. Once the batch grows, the same fixed overhead is shared across many puzzles: at batch size 8, the enhanced path falls to 3.88×10^{-5} seconds per puzzle, already about $2.6\times$ faster than the CPU baseline; at batch size 128, it reaches 3.85×10^{-6} seconds per puzzle, about $26.7\times$ faster than the CPU baseline and $43.4\times$ faster than the original CUDA path.

For $s = 50, k = 15$, each trial has $50^3 = 125,000$ entries in its 3D graph, so the GPU receives substantially more independent work per batch. Even at batch size 1, the enhanced path (3.15×10^{-4} seconds per puzzle) is already about $4.3\times$ faster than the CPU baseline (1.35×10^{-3}), although it is still slower than the original per-puzzle CUDA path (2.21×10^{-4}). From batch size 2 onward,

the enhanced path is faster than both baselines, and at batch size 128 it reaches 6.18×10^{-6} seconds per puzzle. That is about $218\times$ faster than the CPU baseline and about $35.7\times$ faster than the original CUDA path. These measurements show that the improvement comes from amortizing fixed GPU costs across a batch and exposing enough work to keep the device busy; they should be interpreted as hardware- and benchmark-specific evidence for the batching design, not as a hardware-independent performance guarantee.

3 The Search Problem

Although parallel computation can accelerate SIMPLIFY, using batched SIMPLIFY in the search for large SUSPs is not straightforward. The main difficulty is that the staged search program of Anderson and Le processes candidate puzzles one at a time [4]. To use batched SIMPLIFY, the search algorithm must collect many compatible puzzles and simplify them together. Finding the right collection point requires changing the whole search workflow around puzzles.

3.1 Anderson and Le Search Program

Anderson and Le proposed a search program for finding larger SUSPs by building on smaller ones, with each layer of this search represented by a stage [4]. At each stage, the program uses iterated local search [17]: to search for a (s, k) -SUSP, it starts from a known $(s-1, k)$ -SUSP, adds one row, and then repeatedly modifies the resulting puzzle. The candidate transformations include small changes, such as changing a single entry, as well as larger moves, such as iterating through all 3^k possible replacements for a row [4]. Each generated puzzle is simplified to compute the number of edges that remain in its 3D graph, and that edge count is then used in fitness computation to guide the search.

Within each stage, queue elements store candidate puzzles together with a heuristic estimate and a gap from the maximum possible fitness. Smaller gap values are better, and gap 0 indicates that the candidate has reached maximum fitness and is therefore a SUSP under the selected checking mode. The stage queue is ordered by this gap, so candidates with the most promising fitness are expanded first. Because the search space is far too large to store explicitly, each queue is bounded; when the queue fills, the least promising candidates are discarded. To avoid repeatedly searching equivalent puzzles, candidate puzzles are canonicalized and stored in an isomorphism cache. In our enhanced implementation, optional Bloom filtering is also available to reduce the memory pressure of this seen set, at the cost of allowing false positives [6].

The most important workflow mechanism is that the program keeps several stages active even after a stage has found a SUSP. Downward closure guarantees that if a (s, k) -SUSP exists, then any subset of its rows of size $s' \leq s$ is also a (s', k) -SUSP [7]. The converse is not guaranteed: finding a (s, k) -SUSP does not imply that ILS will find a $(s+1, k)$ -SUSP from that particular puzzle. To avoid turning the search into a greedy chain from one discovered puzzle to

the next, Anderson and Le keep multiple stages active in parallel. The largest active stage is the frontier stage, and the smaller stages act as substages that continue searching for alternative SUSPs to feed into the stages above them. The current implementation uses four active stages, namely the frontier stage and three substages. When the frontier stage succeeds and the search advances to the next size, the stage window shifts upward, the smallest substage is deactivated, and a new frontier stage is created from the previous frontier.

3.2 CPU Parallelism and the Batching Bottleneck

Anderson and Le use OpenMP to exploit CPU multithreading throughout this workflow [4]. Multiple worker threads process candidate puzzles concurrently: a worker selects a stage, removes a puzzle from that stage’s frontier, simplifies and scores generated candidates, inserts neighboring puzzles back into bounded queues, and then continues with another puzzle. A main coordination role is typically handled by thread 0, which monitors progress, displays status, checks restart conditions, and helps coordinate shared search state.

Algorithm 2 summarizes this OpenMP staged ILS workflow. The algorithm shows the key batching obstacle in the original program: each worker evaluates generated candidates immediately inside PROCESSCANDIDATE, so puzzles naturally arrive at SIMPLIFY one at a time.

Algorithm 2 OpenMP Staged Iterated Local Search

Input: Known SUSEP P_0 , target size $s_{\max}$, width k , fitness tester F

Output: A $(s_{\max}, k)$ -SUSEP, if found

```

1: function STAGEDILS( $P_0, s_{\max}, k, F$ )
2:    $N \leftarrow 4, f \leftarrow N - 1$ 
3:    $S[0 \dots N - 1] \leftarrow \perp$ 
4:    $S[f] \leftarrow \text{INITSTAGE}(P_0, F)$ 
5:   for  $s \leftarrow |P_0|$  to  $s_{\max}$  do
6:     while  $\text{EMPTY}(S[f].\text{results})$  do
7:       omp parallel
8:         for all OpenMP threads  $t$  do
9:           if  $t = 0$  then
10:            MAINTAIN( $S, f$ )
11:           else
12:             $i \leftarrow \text{PICKSTAGE}(S, f)$ 
13:            if  $i \neq \perp$  then
14:              WORKERSTEP( $S[i], F$ )
15:         end omp parallel
16:       if  $\text{STALLED}(S[f])$  then
17:         RESTART( $S[f]$ )

```

```

18:    $P \leftarrow \text{POPRESULT}(S[f])$ 
19:    $\text{VERIFYANDLOG}(P)$ 
20:    $A \leftarrow \text{NEWSTAGE}(s + 1, k, F, S[f - 1])$ 
21:    $\text{SLIDEWINDOW}(S, A)$ 
22:   return  $P$ 
23: function  $\text{WORKERSTEP}(A, F)$ 
24:    $(Q, g, \tau) \leftarrow \text{POPFONTIER}(A)$ 
25:   if  $Q = \perp$  then
26:     return
27:   if  $g = 0$  then
28:      $\text{PUSHRESULT}(A, Q)$ 
29:   for all  $Q' \in \text{NEIGHBORS}(Q, \tau)$  do
30:      $\text{PROCESSCANDIDATE}(A, Q', F)$ 
31: function  $\text{PROCESSCANDIDATE}(A, Q, F)$ 
32:   omp critical  $\text{new} \leftarrow \text{MARKIFNEW}(A.\text{seen}, Q)$ 
33:   if  $\neg \text{new}$  then
34:     return
35:    $r \leftarrow |\text{SIMPLIFY}(H_Q)|$ 
36:    $(h, h_{\max}) \leftarrow \text{FITNESS}(Q, r, F)$ 
37:    $g \leftarrow h_{\max} - h$ 
38:   if  $g = 0$  then
39:     omp critical  $\text{PUSHRESULT}(A, Q)$ 
40:   else
41:     omp critical  $\text{BOUNDEDENQUEUE}(A, Q, g)$ 

```

OpenMP is effective for this program. In practice, the CPU workers can keep the available CPU threads close to full utilization during long searches, and OpenMP critical regions provide a straightforward way to protect shared resources such as stage queues, statistics, result lists, and the isomorphism cache. This makes the original CPU-based searcher a strong baseline.

However, the number of CPU threads is limited and is much smaller than the amount of fine-grained parallelism available on a GPU. Since the search may simplify hundreds of millions or billions of candidate puzzles, CPU-side simplification remains a bottleneck. Thus, although the workflow is efficient on a multicore CPU, it still leaves substantial room for GPU acceleration.

3.3 Enhanced Search Program with Batched GPU Processing

To identify where batched SIMPLIFY can be applied inside the search, we first look more closely at the expansion stage used by Anderson and Le [4]. Their iterative local search generates neighbors of each candidate puzzle through four main transformations: (i) brute-force last-row generation, (ii) active row replacement, (iii) single-cell derivative mutation, and (iv) row/column element permutation.

The first transformation tries every possible value of one new row and adds that row to the puzzle. It is used when a known $(s - 1, k)$ -SUSP is fed into

the next stage, whose target is a (s, k) -SUSP. Because each of the k entries has three possible values, this transformation creates 3^k candidates from one input puzzle. Active row replacement also creates 3^k candidates: it chooses one active row of the current puzzle and replaces only that row with every possible row of width k . The last two transformations are thinner local moves used when a puzzle is already close to the target fitness. A derivative mutation changes one cell at a time, and the row/column permutation path applies one non-identity value permutation to one row or column at a time. In both cases, the number of candidates is only linear, up to a small constant factor, in the puzzle dimensions.

The first two transformations are therefore natural places for batched SIMPLIFY. They start from one puzzle and produce a dense set of same-shape candidates. For $k \geq 8$, the 3^k candidates are more than enough to fill our fixed `TRIALS = 256` GPU batches, exposing enough parallel work to improve occupancy and help the GPU hide memory latency, which is a standard concern in CUDA optimization [20].

3.4 Batched Workflow and Memory Tradeoff

Using batched SIMPLIFY requires a change in the search workflow. In the enhanced program, we change this order for the two dense candidate streams. Instead of simplifying each puzzle immediately, the program first stores the candidates generated by brute-force last-row generation or active row replacement in a local vector. The candidates are then split into chunks of at most `TRIALS = 256` puzzles. Each chunk becomes one `GpuTask`, a structure that packs all the data into one big buffer; the GPU returns one simplified TDM count per puzzle, and the search code stores that count on the corresponding `Puz` object before calling the ordinary frontier insertion path. The fitness tester then uses the already-computed simplification result instead of recomputing SIMPLIFY on the CPU.

The main tradeoff is memory usage and bandwidth. Storing all candidates from a dense transformation requires host-side memory and bandwidth to prepare, filter, and transfer the batch to the GPU. In particular, the enhanced implementation uses pinned host memory, which is allocated with `cudaMallocHost` and remains page-locked on the host. Page-locked buffers are important for efficient asynchronous host-device transfers, but they also reduce the operating system’s freedom to page memory under pressure [20]. In exchange, this fixed pinned allocation supports the asynchronous workflow used by the GPU path: CPU workers can prepare future work while GPU workers execute submitted batches. Since the batch size is fixed, the pinned-memory footprint is also fixed. Once the machine has enough memory for the whole search, the workflow is stable, and we trade additional memory for higher throughput in the overall search program.

Table 1: Expansion sources and fitness paths in the enhanced staged searcher.

Source in expansion	Fitness path
brute-force last-row generation	batched GPU simplification
active-row replacement	batched GPU simplification
single-cell derivative mutation	immediate normal fitness
row/column element permutation	immediate normal fitness

3.5 Asynchronous Producer-Consumer Workflow

With GPU support, the search program has a clearer role for each component. The CPU is still the main part of the search. It manages the frontier queues, keeps the staged search running, and uses OpenMP so that many CPU threads can explore candidate puzzles from the queues at the same time [4,21]. The GPU is used as a specialized helper for SIMPLIFY: it should simplify as many candidate puzzles as possible so that the CPU can expand the search faster.

This separation creates a synchronization problem. If every CPU worker is allowed to launch GPU kernels freely, then many workers may submit batches at nearly the same time. The CUDA runtime and driver can queue this work, but the submitting CPU worker still needs the simplification results before it can finish scoring and inserting those candidates. Thus a worker can become idle while waiting for its batch to return, even though many other batches are still waiting to be processed. The goal of the enhanced workflow is to keep the CPU workers and GPU busy at the same time.

To coordinate this workflow, we use a producer-consumer pipeline. The basic unit of GPU work is a **GpuTask**. A **GpuTask** contains all information needed to simplify one batch of at most `TRIALS = 256` puzzles: the serialized puzzle data, puzzle dimensions, result space, the identity of the CPU worker that created the task, the task id, and other small metadata. Packaging the batch this way makes the GPU path modular. Once a task is created, the rest of the program can treat it as one object moving through the pipeline.

The **GpuTask** objects are stored in a fixed pinned host-memory region. At search startup, the program allocates one large contiguous buffer, with enough space for each CPU worker to own two task slots. For worker i , the two slots are $2i$ and $2i + 1$. Each worker also stores local metadata telling it which of its two slots should be used for the next task. This creates private double buffering for each worker: worker i can write work only into slot $2i$ or slot $2i + 1$, and no other worker writes into those slots.

After a worker finishes preparing a **GpuTask** and places it in one of its slots, it pushes the slot id into an atomic queue. The queue uses atomic operations to give the slot ids a well-defined ordering [12]. Even if two CPU workers submit ids at almost the same time, one id is placed before the other. The worker does not need to launch the GPU kernel itself. After submitting the slot id, it can return to other search work assigned by the CPU manager, such as expanding another puzzle and preparing the next batch.

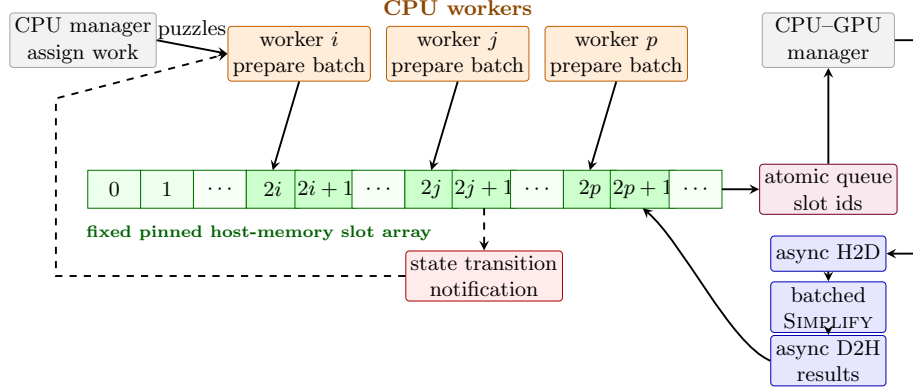


Fig. 6: Producer-consumer workflow for batched GPU simplification. CPU workers build `GpuTask` objects in their own pinned memory slots and submit only slot ids to an atomic queue. A CPU-GPU manager consumes slot ids, launches the asynchronous GPU copy and simplification work, writes results back into the same slots, and notifies the corresponding workers when each task changes state.

In addition to the CPU coordination used in the staged search, the enhanced program has a CPU-GPU coordination layer for GPU work. This CPU-GPU manager repeatedly reads the first available slot id from the atomic queue. That id tells it which memory slot contains the next task and which worker created it. The manager reads the task metadata and then marks the task as processing. This changes the task state from `Ready` to `Processing` using an atomic compare-and-swap on the task's `std::atomic` state field of type `GpuTaskState`. It then calls `notify_all()` on the task's C++ condition variable to notify the owner worker. In other words, the manager uses the condition variable mechanism together with an atomic state transition to tell the worker that the task is now being processed [12]. This notification is important: once the slot is marked busy, the owner worker will not overwrite it while the GPU is reading or writing that slot. The manager can then launch the batched GPU simplification for that `GpuTask`.

In our experiments, two slots per worker were enough for each worker to almost always have a free place to put its next `GpuTask`. If the GPU becomes slower than the CPU producers and both slots of a worker are busy, then that worker has to wait. Concretely, the worker reads the `GpuTaskState` of its slots. If no slot is available, it waits with `slot.cv.wait_for(...)` until one slot becomes `Done` or `Empty`. This is the back-pressure point of the pipeline. It prevents workers from creating unbounded GPU work and protects the fixed pinned-memory allocation.

When the GPU finishes a `GpuTask`, the results are written back to the same slot. The CPU-GPU manager then calls `markDone()` on the task, which changes the state from `Processing` to `Done` and again calls the condition variable's `notify_all()` method to tell the owner worker that the task has completed [12]. The worker may currently be preparing another batch, so it can finish that local

work first and then return to the completed task. At that point, it reads the simplified counts, continues the ordinary search logic, performs the needed isomorphism checks, inserts promising candidates into the frontier, and marks the slot empty again. The worker is then ready to receive more work from the CPU manager.

Each worker also has its own small work queue. Manager threads add items to these queues instead of forcing all workers to share one piece of mutable state. The CPU manager can add a task such as “expand this puzzle,” while the CPU-GPU manager can add a task such as “this `GpuTask` is finished; complete the insertion path.” Shared structures are protected only at the points where sharing is necessary, using mechanisms such as OpenMP critical regions, atomics, mutexes, and notification primitives [21,12].

The resulting workflow forms a pipeline. The CPU manager assigns search work, the CPU workers prepare batches in their own memory slots, the atomic queue orders the ready slots, and the GPU acts as the machine that performs batched SIMPLIFY. Because each worker has its own slots, the workers rarely need to share memory directly with each other. This reduces common multithreading hazards such as accidental overwrite, and it gives the CPU and GPU a clear asynchronous boundary. The pinned-memory slots make host-device copy and GPU computation easier to overlap, helping both the CPU workers and the GPU stay close to full utilization [20].

3.6 Multiple-GPU Environment

The pipeline makes the multi-GPU extension for the search program straightforward to implement. Each GPU worker repeatedly takes the next available batch from the shared queue, runs it on its own device, and returns the results to the CPU worker waiting for that batch.

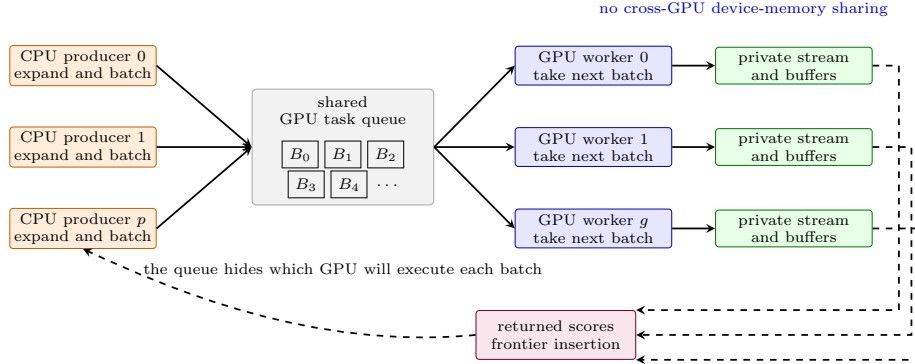


Fig. 7: Producer-consumer workflow for multiple GPUs. Each GPU worker returns scores for frontier insertion using its own private device resources.

The expected speedup is therefore bounded by how much batchable work the CPU producers can create. When dense transformations dominate, the queue

can keep several GPUs busy and the simplification rate should increase with the number of devices. However, the search in our current environment spends more time in thin local moves, isomorphism checks, or frontier management due to limitations of the CPU. The GPUs may wait for new batches and cannot operate at full utilization.

3.7 Search Benchmark

Figure 8 reports total simplifications per second. The original staged values are preserved from previous recorded figure data. The enhanced staged values for (14, 6)-puzzle, (23, 7)-puzzle, (35, 8)-puzzle, and (52, 9)-puzzle are measured in the same experimental environment described above. The (64, 15)-puzzle point is an enhanced-only capability point with no original staged baseline because the original program cannot practically search at that puzzle size.

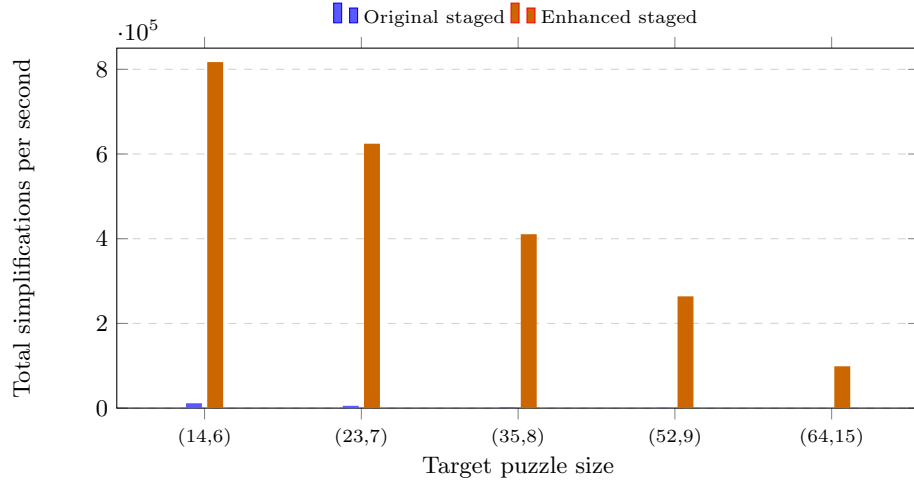


Fig. 8: Total simplifications per second for the original staged searcher and the enhanced staged searcher.

Table 2: Total simplification throughput for the staged search benchmark. The (64, 15)-puzzle entry is enhanced-only because no original staged run was recorded for that target.

Target	Original staged	Enhanced staged	Ratio
(14, 6)-puzzle	11466.97	817053.49	71.25×
(23, 7)-puzzle	5425.86	624315.15	115.06×
(35, 8)-puzzle	2230.51	410595.08	184.08×
(52, 9)-puzzle	930.00	263894.16	283.76×
(64, 15)-puzzle	—	98753.78	—

The benchmark supports two conclusions. First, batching inside the search loop greatly increases simplification throughput in the staged search benchmark. Relative to the preserved original staged baseline, the enhanced staged search records $71.25\times$, $115.06\times$, $184.08\times$, and $283.76\times$ higher total simplification throughput on (14,6)-puzzle, (23,7)-puzzle, (35,8)-puzzle, and (52,9)-puzzle, respectively. The (64,15)-puzzle value reports 98753.78 total simplifications per second for the enhanced staged search, but no ratio is computed because the original staged search was not run for that configuration. Second, the enhanced staged search is capable of searching for SUSPs at larger sizes where the original search program is not practical in our setting, especially when $k \geq 10$.

4 Other Improvements

In addition to the main GPU batching pipeline, the enhanced search program also adds two supporting features: checkpointing for long searches and an optional Bloom filter for the isomorphism cache. These features make the implementation more practical when the search must run for a long time and the number of explored candidates becomes very large.

4.1 Checkpointing

The staged search can run for hours or days, so losing the whole run after an interruption is expensive. To address this, the enhanced program can save the current search state into a binary checkpoint file. The checkpoint is triggered by `SIGINT`. When the user presses Ctrl-C, the signal handler only sets a global atomic flag, `should_checkpoint`. The OpenMP search loop checks this flag between units of work. Thus the program does not stop in the middle of modifying a queue or in the middle of a GPU batch. Instead, worker threads finish their current step and then leave the parallel search region.

This cooperative stopping point is important because the GPU path is asynchronous. A CPU worker may have already submitted one or more `GpuTask` slots to the CPU-GPU manager. In the enhanced stage implementation, the worker keeps an `outstanding` count for its submitted slots and waits until every outstanding slot reaches `Done`. It then copies the simplified counts back into the candidate puzzles, continues the ordinary frontier-insertion logic, and marks the slots `Empty`. Therefore, by the time the worker returns from its current search step, the GPU results that it submitted have already been folded back into the normal CPU stage state.

After all OpenMP workers have reached this checkpoint boundary, the program serializes the global `Search` object. This snapshot contains the search mode, the strong flag, the target size, the current puzzle, the current row size, and the active stage window. Each active `ILSStage` serializes its statistics, its bounded multilevel feedback queues, and any completed results waiting to be fed to later stages. Each queue element stores its fitness gap, the candidate puzzle, and the move type that created it. The checkpoint also stores the canonical

isomorphism-cache state, the name of the fitness tester, and the Bloom-filter state when Bloom filtering is enabled.

Restoring follows the reverse path. If the enhanced search receives a `.dat` file as its optional start file, it deserializes the `Search` object, rebuilds the active stages and their previous stage pointers, restores the isomorphism cache, restores the Bloom filter state, and recreates the appropriate fitness tester. The program then resumes from the saved row size and current stage instead of starting again from the initial puzzle. If a checkpoint already contains Bloom-filter data, command-line Bloom options are ignored on resume so that the resumed run follows the same duplicate-detection state that was saved.

4.2 Bloom Filtering

The search avoids repeated work by canonicalizing each candidate puzzle and checking whether an isomorphic puzzle has already been seen. The exact cache stores canonical graph strings in a map. This is accurate, but it can consume a large amount of memory when the search explores many candidates. A Bloom filter gives a different trade-off: it stores membership information in a compact bit array, using several hash functions, and allows false positives but no false negatives [6]. In this setting, a false positive means that a new candidate may be treated as already seen and skipped. Therefore Bloom filtering can reduce memory usage, but it can also make the heuristic search more aggressive. This does not mean that a Bloom-filtered run cannot find a SUSP. The search is randomized and usually has many different paths to promising puzzles, so even if one useful candidate is skipped, a long enough run may still reach a SUSP through another candidate.

For a Bloom filter with m bits, k hash functions, and n inserted elements, the usual false-positive probability is

$$p \approx \left(1 - e^{-kn/m}\right)^k.$$

For a target false-positive rate p and an expected number of insertions n , the standard parameter choices are

$$m = -\frac{n \ln p}{(\ln 2)^2}, \quad k = \frac{m}{n} \ln 2.$$

Thus lowering the requested false-positive rate increases the memory requirement, and increasing the expected number of elements increases the memory requirement almost linearly. If the run inserts many more than the expected number of elements, the actual false-positive rate will grow beyond the requested value.

In the enhanced implementation, Bloom filtering is used in the isomorphism-check routine. The puzzle is first canonicalized in the same way as the exact isomorphism cache. If Bloom filtering is disabled, the canonical graph is stored and queried in the exact map. If Bloom filtering is enabled, the canonical graph

bytes are checked against the Bloom filter and optionally inserted there instead. This keeps the isomorphism interface the same for the search loop while replacing the large exact seen set with a much smaller approximate structure.

The feature is controlled by the optional command-line argument `--bloom`. The argument may also include an expected count and false-positive rate, for example `--bloom=100000000,0.01`. Without this argument, Bloom filtering is disabled. If the user passes `--bloom` without parameters, the program uses the current default of 10^8 expected canonical puzzles and false-positive rate 0.01. With these defaults, the formula above gives about 9.59×10^8 bits, or roughly 114 MiB of filter storage, and the implementation uses six hash probes. Increasing the expected count or lowering the false-positive rate asks for a larger filter; using a larger false-positive rate uses less memory but skips more candidates in expectation. For this reason, Bloom filtering is mainly useful in memory-constrained environments where the exact isomorphism map would become too large. When enough memory is available, the exact hash map is still the recommended default because it does not discard unseen candidates.

5 Conclusions

This work applies GPU computing to the search for simplifiable SUSPs, building on the staged search framework and CUDA simplification path of Anderson and Le [4].

The enhanced implementation follows this idea at two levels. First, the SIMPLIFY computation is rewritten around batched GPU data structures: packed puzzle rows, persistent device buffers, a fixed device-side simplification schedule, and GPU-side reductions. This removes repeated allocation and many host-device synchronization points. Second, the staged search loop is changed so that dense transformations produce `GpuTask` batches. A producer-consumer pipeline with pinned host-memory slots, an atomic slot queue, and CPU-GPU coordination lets CPU workers continue preparing search work while the GPU processes earlier batches asynchronously.

The benchmarks show that, in this implementation, this workflow gives a large practical speedup. In isolated simplification tests, the enhanced batched path is much faster than both the CPU path and the original per-puzzle CUDA path once the batch is large enough to amortize fixed GPU overhead. Inside the real staged search, the enhanced implementation records much higher total simplification throughput on the measured targets, including $71.25\times$, $115.06\times$, $184.08\times$, and $283.76\times$ improvements over the preserved original staged baseline for (14,6)-puzzle, (23,7)-puzzle, (35,8)-puzzle, and (52,9)-puzzle. The larger (64,15)-puzzle run also shows that the new program can keep searching in regimes where the original workflow cannot.

This does not yet give a new bound on the matrix multiplication exponent. It is instead an implementation step toward making larger searches for simplifiable SUSPs feasible. The remaining bottlenecks are now less concentrated in SIMPLIFY itself and more about the underlying theory. Future work should there-

fore focus more on theoretical improvements, including new tools for simplifying puzzles and new directions for searching for simplifiable SUSPs or SUSPs more broadly.

Acknowledgments

The author thanks Professor Matthew Anderson for his direct support, guidance, and feedback throughout this project. The author also thanks Professor Anderson and Vu Le for providing the foundation for this work through their paper on efficiently-verifiable strong uniquely solvable puzzles.

References

1. Alman, J., Duan, R., Vassilevska Williams, V., Xu, Y., Xu, Z., Zhou, R.: More asymmetry yields faster matrix multiplication (2024). <https://doi.org/10.48550/arXiv.2404.16349>, <https://arxiv.org/abs/2404.16349>, in SODA 2025
2. Alman, J., Vassilevska Williams, V.: A refined laser method and faster matrix multiplication. In: Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms (SODA). pp. 522–539. SIAM (2021). <https://doi.org/10.1137/1.9781611976465.32>
3. Anderson, M., Ji, Z., Xu, A.Y.: Matrix multiplication: Verifying strong uniquely solvable puzzles. In: Pulina, L., Seidl, M. (eds.) Theory and Applications of Satisfiability Testing (SAT). pp. 464–480. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-51825-7_32
4. Anderson, M., Le, V.: Efficiently-verifiable strong uniquely solvable puzzles and matrix multiplication (2023). <https://doi.org/10.48550/arXiv.2307.06463>, <https://arxiv.org/abs/2307.06463>
5. Barnat, J., Bauch, P., Brim, L., Ceska, M.: Computing strongly connected components in parallel on CUDA. In: 2011 IEEE International Parallel and Distributed Processing Symposium. pp. 544–555. IEEE (2011). <https://doi.org/10.1109/IPDPS.2011.59>
6. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970). <https://doi.org/10.1145/362686.362692>
7. Cohn, H., Kleinberg, R., Szegedy, B., Umans, C.: Group-theoretic algorithms for matrix multiplication. In: 46th Annual IEEE Symposium on Foundations of Computer Science (FOCS). pp. 379–388 (2005). <https://doi.org/10.1109/SFCS.2005.39>
8. Cohn, H., Umans, C.: A group-theoretic approach to fast matrix multiplication. In: 44th Annual IEEE Symposium on Foundations of Computer Science (FOCS). pp. 438–449 (2003). <https://doi.org/10.1109/SFCS.2003.1238217>
9. Coppersmith, D., Winograd, S.: Matrix multiplication via arithmetic progressions. Journal of Symbolic Computation **9**(3), 251–280 (1990). [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2)
10. Davie, A.M., Stothers, A.J.: Improved bound for complexity of matrix multiplication. Proceedings of the Royal Society of Edinburgh Section A: Mathematics **143**(2), 351–369 (2013)

11. Duan, R., Wu, H., Zhou, R.: Faster matrix multiplication via asymmetric hashing (2022). <https://doi.org/10.48550/arXiv.2210.10173>, <https://arxiv.org/abs/2210.10173>
12. International Organization for Standardization: ISO/IEC 14882:2020: Programming languages – C++ (2020), international standard
13. Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., et al.: In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA). pp. 1–12 (2017). <https://doi.org/10.1145/3079856.3080246>
14. Karp, R.M.: Reducibility among combinatorial problems. In: Miller, R.E., Thatcher, J.W. (eds.) Complexity of Computer Computations, pp. 85–103. Plenum Press, New York (1972)
15. Kung, H.T.: Why systolic architectures? Computer **15**(1), 37–46 (1982). <https://doi.org/10.1109/MC.1982.1653825>
16. Le Gall, F.: Powers of tensors and fast matrix multiplication. In: 39th International Symposium on Symbolic and Algebraic Computation (ISSAC). pp. 296–303. ACM (2014). <https://doi.org/10.1145/2608628.2608664>
17. Lourenço, H.R., Martin, O.C., Stützle, T.: Iterated local search (2001), <https://arxiv.org/abs/math/0102188>
18. Nickolls, J., Buck, I., Garland, M., Skadron, K.: Scalable parallel programming with CUDA. Queue **6**(2), 40–53 (2008). <https://doi.org/10.1145/1365490.1365500>
19. NVIDIA Corporation: CUB: CUDA cooperative primitives (2026), <https://nvidia.github.io/cccl/cub/>, accessed 12 June 2026
20. NVIDIA Corporation: CUDA C++ Best Practices Guide (2026), <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>, version 13.3, accessed 17 June 2026
21. OpenMP Architecture Review Board: OpenMP application programming interface version 5.1 (2020), <https://www.openmp.org/spec-html/5.1/openmp.html>
22. Strassen, V.: Gaussian elimination is not optimal. Numerische Mathematik **13**(4), 354–356 (1969). <https://doi.org/10.1007/BF02165411>
23. Williams, V.V.: Multiplying matrices faster than Coppersmith-Winograd. In: 44th Annual ACM Symposium on Theory of Computing (STOC). pp. 887–898. ACM (2012). <https://doi.org/10.1145/2213977.2214056>